

무료 코딩 에이전트 설치 가이드

GitHub Copilot과 OpenCode는 **VS Code**에서 사용하고, Antigravity와 TRAE는 별도 앱(IDE/워크스테이션)으로 실행한다.

1. 설치 전 공통 준비물

먼저 아래 항목을 준비한다. 인터넷 연결과 프로그램 설치 권한은 네 도구에 모두 필요하다. 나머지는 사용할 도구에 따라 준비한다.

네 도구에 공통으로 필요한 항목

항목	준비 방법	확인 방법
인터넷 연결	웹사이트 접속과 로그인, 프로그램 다운로드가 가능한 네트워크 사용	브라우저에서 설치 페이지 접속
웹 브라우저	Edge, Chrome, Firefox 등 설치	로그인 페이지가 열리는지 확인
프로그램 설치 권한	개인 PC 또는 프로그램을 설치할 수 있는 실습 PC 사용	설치 파일 실행 가능 여부 확인
프로젝트 작업 폴더	파일을 저장하고 수정할 권한이 있는 폴더 준비	VS Code, Antigravity 또는 TRAE에서 폴더 열기
프로젝트 규칙 파일	작업 폴더 루트에 <code>AGENTS.md</code> 작성 (6절)	도구에 "이 프로젝트 규칙을 요약해줘"라고 물어 확인

도구별로 필요한 항목

사용할 도구	추가 준비물	설치·가입 방법	확인 방법
GitHub Copilot	VS Code, 개인 GitHub 계정, 학생 신분 증빙	VS Code 설치, GitHub 가입	VS Code 실행, GitHub 로그인
Antigravity	Google 계정	Google 계정 준비	브라우저 로그인
TRAE	TraeWork(또는 TraeCode) 데스크톱 앱	TRAE 개요 참고 후 설치	앱 실행 후 프로젝트 폴더 열기
OpenCode	VS Code, Node.js LTS, 터미널	Node.js LTS 설치	<code>node --version</code> , <code>npm --version</code>

Git은 설치 자체의 필수 조건은 아니지만, 수업 저장소를 내려받고 변경 이력을 관리하려면 설치하는 편이 낫다. [git-scm.com](#)에서 설치한 뒤 `git --version`으로 확인한다. OpenCode에서 로컬 모델을 쓸 학생만 Ollama와 모델 저장 공간을 추가로 준비한다.

도구 요약

도구	무료 한도	주 용도	난이도
GitHub Copilot Student	코드 완성 무제한 + 월 200 AI Credits	일상 코딩 완성	신청 까다로움
Antigravity IDE	Gemini 기반, 5시간마다 한도 갱신	복잡한 멀티스텝 작업	설치 쉬움
TRAE (TraeWork / TraeCode)	요금·한도는 정책에 따라 변동 (앱/공식 문서에서 확인)	에이전트 기반 멀티스텝 작업, 코드/문서 산출	설치 보통
OpenCode	Zen 무료 모델 + 로컬 모델(Ollama)	에이전트 작업, 코드 분석	모델 설정 까다로움

- ChatGPT, Claude 유료 사용 가능

2. GitHub Copilot Student (신청 → 승인 → VS Code 설치)

2-1. 왜 까다로운가

- GitHub Education Student Developer Pack을 먼저 승인받아야 Copilot을 쓸 수 있다

- 학생 신분 증명을 통과해야 하고, 승인까지 수 시간~2-3일이 걸린다
- 서류가 불분명하면 거절되고 다시 신청해야 한다
- 2026년 4-6월에 신규 가입이 일시 중단된 적이 있다. 2026년 6월 17일부터 재개되어 현재 신규 가입이 가능하다. 다만 정책이 바뀔 수 있으므로 신청 페이지의 최신 공지를 확인한다

2-2. 자격 조건

- 학위·졸업장 수여 과정에 재학 중
- 만 13세 이상
- 개인 GitHub 계정 보유 (조직 계정 불가)

2-3. 신청 절차 (단계별)

① GitHub 계정 준비

- github.com에서 개인 계정을 만든다 (이미 있으면 건너뛰)
- 프로필 이름을 실명으로 설정한다 — 서류의 이름과 일치해야 한다

② Student Developer Pack 신청

1. education.github.com/pack 접속
2. "Sign up for Student Developer Pack" 또는 "Get benefits" 클릭
3. 역할 선택: **Student**
4. 학교 선택: 학교 이름을 검색하거나 직접 입력

③ 학생 신분 증명 (2가지 방법)

○ 방법 A: 학교 이메일 인증 (가장 빠름)

- `@university.ac.kr`, `@univ.edu` 같은 학교 이메일을 GitHub 계정에 추가한다
- Settings → Emails → 학교 이메일 추가 → 인증 메일 확인
- 신청 시 학교 이메일을 선택하면 자동으로 통과하는 경우가 많다

○ 방법 B: 서류 업로드 (학교 이메일이 없거나 인증이 안 될 때)

서류 종류	영문 표기	주의사항
학생증 사진	Student ID	이름 + 학교명 + 현재 재학 기간/날짜 가 보여야 한다
수업 시간표	Class schedule	이름, 학교명, 현재 학기 수업이 표시되어야 한다
성적 증명서	Transcript	이름과 현재 재학 상태가 보여야 한다
재학증명서	Enrollment verification letter	학교에서 발급한 공식 서류. 영문이면 더 좋다
등록금 납부 영수증	Tuition receipt	현재 학기 납부 내역 (이름·학교·날짜 필수)
학교 포털 스크린샷	Screenshot of student portal	본인 이름 + 현재 재학 상태가 보이는 화면 캡처

서류 업로드 시 거절을 피하려면:

- 글자가 선명하게 보여야 한다 — 흐리거나 잘리면 거절된다
- GitHub 프로필 이름과 서류의 이름이 일치해야 한다
- 현재 학기/연도가 보여야 한다 — 오래된 서류는 거절된다
- 이미지 파일로 올린다 — PDF는 지원이 안된다. 즉 위의 서류를 받으면 png 파일 등으로 chatgpt 등에서 변환한다.

④ 승인 대기

- 보통 수 시간 ~ 2-3일
- 거절되면 이메일에 이유가 오고, 다른 서류로 다시 신청할 수 있다

2-4. Copilot Student 혜택 (승인 후)

항목	내용
코드 완성	무제한
AI Credits	월 200 (채팅·에이전트용)
모델 선택	Auto만 가능 (수동 선택 불가)

2-5. VS Code에서 Copilot 활성화

1. VS Code를 연다
2. 확장(Extensions) 탭에서 **"GitHub Copilot"** 검색 → 설치
3. 좌측 하단 사람 아이콘 → **Sign in with GitHub** → 로그인
4. 상태 바에 Copilot 아이콘이 나타나면 활성화된 것이다
5. 확인: github.com/settings/copilot 에서 Copilot Student 상태 확인

3. Antigravity IDE (Google)

3-1. 설치 (Windows)

1. antigravity.google/download에서 **Windows** 버전 다운로드 (x64)
2. 다운로드한 `.exe` 실행
3. Windows Defender SmartScreen이 뜨면 "추가 정보" → "실행" 클릭
4. 설치 완료 후 실행 → **Google 계정으로 로그인**
5. 테마 선택 및 에이전트 정책(터미널 실행, 코드 리뷰 등) 설정

3-2. 현재 버전 (2026년 8월 기준)

항목	버전
Antigravity 2.0	v2.8.1
Antigravity IDE	v2.5.5
지원 OS	Windows 10/11 64-bit

3-3. VS Code와의 관계

- Antigravity는 **독립 IDE**이므로 VS Code 안에서 실행하는 것이 아니다
- VS Code 프로젝트와 같은 폴더를 열어 병행 사용할 수 있다
- Gemini 기반이라 Google 계정만 있으면 별도 API 키 없이 바로 쓸 수 있다

3-4. CLI 설치 (선택)

PowerShell에서:

```
irm https://antigravity.google/cli/install.ps1 | iex
```

4. TRAE (TraeWork / TraeCode)

참고: TRAE는 IDE(TraeCode)와 AI 워크스테이션(TraeWork) 등 여러 제품군으로 구성된다. 상황에

따라 하나만 설치해도 된다. (TRAE 개요)

4-1. 무엇을 할 수 있나

- 자연어로 목표를 주면, 작업을 쪼개서 계획하고(Plan) 실행(Build)하는 **에이전트(Agent)** 중심의 흐름을 제공한다
- 코드뿐 아니라 문서/리포트 같은 산출물 생성, 프로젝트 맥락 기반의 수정 작업에 적합하다

4-2. 설치 (Windows)

1. 공식 문서의 다운로드 안내에서 TraeWork(또는 TraeCode) 데스크톱 앱을 설치한다: <https://docs.trae.cn/>
2. 앱을 실행한 뒤, 과제/프로젝트 폴더를 연다
3. Code/IDE 모드에서 채팅 또는 Agent 기능으로 작업을 진행한다
4. \$3, \$10 등 저렴한 가격도 있다.

4-3. 처음 사용할 때 팁

- 처음엔 "현재 폴더에서 README 를 읽고 해야 할 일을 정리해줘" 같은 작은 요청부터 시작한다
- 계획(Plan)과 실행(Build)이 나뉘는 흐름이 있으면, 실행 전에 계획을 먼저 확인한다

5. OpenCode (설치 → 모델 연결 → VS Code 연동)

5-1. 설치부터 모델 선택까지 한 흐름으로

VS Code 터미널에서 `opencode` 를 실행하면 확장이 자동 설치되고, 그 안에서 모델 연결·선택까지 한 번에 끝난다. npm으로 `opencode` 명령만 먼저 설치하면 된다.

사전 준비:

- [Node.js LTS](#) 설치 (18 이상)
- VS Code에 `code` 명령이 PATH에 등록되어 있어야 한다
 - 등록 방법: VS Code에서 `Ctrl+Shift+P` → "Shell Command: Install 'code' command in PATH" 실행

단계 1: **opencode CLI 설치** (PowerShell에서 1회만)

```
npm install -g opencode-ai@latest
```

- 확인: `opencode --version`
- Scoop 사용자는 `scoop install opencode`, Chocolatey 사용자는 `choco install opencode -y` 도 가능

단계 2: VS Code에서 실행 + 확장 자동 설치

1. VS Code를 연다
2. 통합 터미널을 연다 (Ctrl + 백틱)
3. 터미널에서 실행:

```
opencode
```

4. VS Code 확장이 **자동으로 설치**된다. 안 되면 확장 마켓플레이스에서 `sst-dev.opencode` 수동 설치

단계 3: 모델 연결 (여기가 까다롭다)

```
/connect
```

- 목록에서 **OpenCode Zen** (또는 `opencode`) 선택
- 브라우저가 열린다 → `opencode.ai`에서 로그인 (GitHub 또는 Google 계정)
- API 키가 나오면 복사 → 터미널에 붙여넣기

단계 4: 무료 모델 선택

```
/models
```

- 목록에서 **"Free"가 붙어 있는 모델**을 선택한다
- 2026년 8월 기준 무료 모델 예시: DeepSeek V4 Flash Free, MiMo-V2.5 Free, Nemotron 계열
- 무료 모델 목록은 수시로 바뀌므로 사용할 때마다 `/models` 로 확인한다

5-2. 자주 쓰는 명령어

명령어	하는 일
<code>/connect</code>	모델 제공자 연결·API 키 등록
<code>/models</code>	사용 가능한 모델 목록 보고 선택
<code>/init</code>	프로젝트 분석 후 <code>AGENTS.md</code> 생성 (처음 한 번 권장 — 6절 참고)
Tab 키	Plan (분석만) ↔ Build (코드 수정) 모드 전환

5-3. VS Code 단축키

단축키	동작
Ctrl + Esc	OpenCode 실행/포커스
Ctrl + Shift + Esc	새 세션 시작
Alt + Ctrl + K	파일 참조 삽입 (@File#L37-42 형식)

6. 프로젝트 규칙 파일 설정 (AGENTS.md · CLAUDE.md · context.md · todo.md)

에이전트는 새 대화를 시작할 때마다 프로젝트를 처음 보는 상태가 된다. "이 폴더가 무슨 프로젝트이고, 실행 명령은 무엇이며, 어디까지 했는지"를 매번 다시 설명하지 않으려면 그 내용을 파일로 남겨야 한다. 이 절에서 만들 파일은 네 개다.

6-1. 두 종류를 구분한다

네 파일은 성격이 다르다. 섞어서 이해하면 "todo.md만 만들어 두면 에이전트가 알아서 읽겠지"라고 오해하게 된다.

종류	파일	어떻게 읽히나
규격 파일	AGENTS.md , CLAUDE.md	도구가 대화 시작 시 자동으로 읽는다. 파일 이름과 위치가 정해져 있다
프로젝트 문서	context.md , todo.md	이름이 자유롭고, 자동으로 읽히지 않는다. 규격 파일 안에 "이 파일을 읽어라"라고 써야 읽는다

6-2. 도구별로 무엇을 읽는가

도구	자동으로 읽는 파일	위치	기본 상태
GitHub Copilot (VS Code)	AGENTS.md	작업 폴더 루트	켜져 있음 (chat.useAgentsMdFile)
OpenCode	AGENTS.md	작업 폴더 루트 (없으면 상위 폴더를 거슬러 탐색)	켜져 있음. /init 이 만들어 준다
Antigravity	AGENTS.md 또는 GEMINI.md	작업 폴더 루트	켜져 있음 (파일당 12,000자 제한)
TRAE	AGENTS.md	작업 폴더 루트	켜져 있음
Claude Code	CLAUDE.md	작업 폴더 루트	켜져 있음. AGENTS.md 는 읽지 않는다

AGENTS.md 하나면 네 도구 중 셋이 그대로 읽는다. Claude Code만 파일 이름이 다르므로 CLAUDE.md 를 따로 두되, 내용을 복사하지 않고 가져다 쓰게 만든다.

참고: VS Code Copilot은 .github/copilot-instructions.md 도 읽는다. 다만 AGENTS.md 가 이미 읽히므로 수업 과제에서는 만들지 않아도 된다. GitHub 웹에서 Copilot coding agent에게 이슈를 맡길 때만 추가로 고려한다.

6-3. AGENTS.md 하나를 원본으로 둔다

같은 내용을 여러 파일에 복사해 두면 한쪽만 고쳐지고 서로 어긋난다. 규칙은 AGENTS.md 한 곳에만 쓰고, CLAUDE.md 는 그것을 불러오게 한다. Claude Code는 @경로 문법으로 다른 파일을 불러올 수 있다.

CLAUDE.md 전체 내용 (두 줄이면 충분하다):

@AGENTS.md

작업 시작 전 context.md와 todo.md를 읽는다.

과제 폴더의 구성은 다음과 같아진다.

```
my-project/
├─ AGENTS.md      # 규칙 원본 - 여기만 고친다
├─ CLAUDE.md      # @AGENTS.md 한 줄
├─ context.md     # 지금 상태
├─ todo.md        # 남은 작업
└─ src/
```

Claude Code를 쓰지 않는다면 `CLAUDE.md` 는 만들지 않아도 된다.

6-4. AGENTS.md에 무엇을 쓰는가

실행 명령, 폴더 구조, 코드 규칙, 금지 사항 네 가지를 적는다. 프로젝트를 처음 맡은 사람에게 30초 동안 설명할 내용이라고 보면 된다.

프로젝트 규칙

이 프로젝트

서울시 따릉이 대여 데이터를 분석해 대여소별 수요를 예측한다.

실행 방법

- 가상환경: ``python -m venv .venv`` 실행 후 ``.venv\Scripts\activate`` (Windows)
- 의존성 설치: ``pip install -r requirements.txt``
- 분석 실행: ``python src/analyze.py``

폴더 구조

- ``data/`` 원본 CSV (수정 금지)
- ``src/`` 분석 코드
- ``results/`` 출력 그림·표

코드 규칙

- 한 함수는 40줄을 넘기지 않는다
- 파일 경로는 ``pathlib.Path``로 다룬다

하지 말 것

- ``data/`` 안의 파일을 덮어쓰지 않는다
- 새 라이브러리를 설치하기 전에 먼저 물어본다

작업 절차

- 작업 시작 전 ``context.md``와 ``todo.md``를 읽는다
- 작업을 끝내면 ``todo.md``의 해당 항목에 체크하고 결과를 한 줄 기록한다

길게 쓴다고 더 잘 지켜지지는 않는다. Claude Code 공식 문서는 200줄 이내를 권하고, Antigravity는 파일당 12,000자에서 자른다. 처음에는 위 분량으로 시작하고, 에이전트가 같은 실수를 두 번 하면 그때 한 줄씩 추가한다.

6-5. context.md와 todo.md — 진행 상황을 넘기는 방법

AGENTS.md에는 잘 바뀌지 않는 규칙을 담고, 자주 바뀌는 진행 상황은 따로 둔다.

파일	담는 것	갱신 시점
context.md	지금 상태, 결정한 사항과 그 이유, 아직 정하지 않은 것	방향이 바뀔 때
todo.md	남은 작업 목록과 완료 표시	작업 하나가 끝날 때마다

todo.md 예시:

```
# 할 일

## 진행 중
- [ ] 대여소별 일별 대여량 집계 (src/aggregate.py)

## 다음
- [ ] 요일·시간대 변수 추가
- [ ] 회귀 모델 1차 적합

## 완료
- [x] 원본 CSV 로드와 결측 확인 - 결측 1.2%, 해당 행 제외로 처리
```

두 파일은 만들어 두기만 해서는 읽히지 않는다. 앞서 AGENTS.md의 "작업 절차"에 넣은 두 줄이 이 파일들을 읽히게 만드는 장치다. 규칙 파일은 자동으로 읽히므로, 그 안에서 다른 파일을 지목하면 에이전트가 따라 읽는다.

이렇게 해 두면 한도가 소진되어 대화가 끊기거나 Copilot에서 Antigravity로 도구를 바꿔도, 새 대화가 todo.md를 읽고 중단 지점부터 이어간다. 도구를 갈아탈 때 규칙 파일을 다시 만들 필요는 없다. 작업 폴더가 같으면 같은 파일을 읽는다.

6-6. 직접 해 보기 (4단계)

1. 과제 폴더 루트에 AGENTS.md를 만들고 6-4의 틀을 채운다
2. 같은 위치에 CLAUDE.md를 만들고 @AGENTS.md를 넣는다 (Claude Code를 쓰지 않으면 건너뛰다)
3. context.md와 todo.md를 만든다. 각각 세 줄로 시작해도 된다
4. 도구를 열고 "이 프로젝트 규칙을 요약해줘"라고 물어본다. AGENTS.md에 쓴 내용이 답에 나오면 읽힌 것이다

4단계에서 규칙 내용이 나오지 않으면 8절 문제 해결을 본다.

7. 네 도구를 돌아가며 쓰는 방법

추천 운영 방식

상황	쓸 도구	이유
평소 코딩 (자동완성)	GitHub Copilot	코드 완성이 무제한이고 VS Code에서 자동으로 동작한다
복잡한 멀티스텝 작업	Antigravity	Gemini 기반으로 한도가 관대하다 (5시간마다 갱신)
코드+문서 함께 만들기 / 에이전트 자동화	TRAE	Agent 중심으로 계획→실행 흐름이 분명하고, 산출물 생성에 강하다
코드 분석·리팩토링	OpenCode (Zen 무료 모델)	에이전트 모드로 파일을 읽고 수정할 수 있다
Copilot Credits 소진 시	OpenCode로 전환	/connect 로 다른 무료 모델을 선택한다
도구를 바꿔 이어서 작업	어느 도구든	AGENTS.md 와 todo.md 가 같은 폴더에 있으면 새 도구가 그대로 이어받는다 (6절)

한도 관리 요약

도구	무료 한도	한도 소진 시
GitHub Copilot Student	완성 무제한 + 월 200 Credits	Credits가 소진되면 채팅·에이전트만 제한. 완성은 계속 된다
Antigravity	5시간마다 갱신	한도 갱신을 기다리거나 다른 도구로 전환한다
TRAE	요금·한도는 정책에 따라 변동	앱/공식 문서에서 요금제·한도를 확인하고, 필요 시 다른 도구로 전환한다
OpenCode Zen	무료 모델별 일일 한도	다른 Free 모델로 전환하거나 로컬 모델(Ollama)을 쓴다

8. 문제 해결

GitHub Copilot

문제	해결
Student Pack 거절됨	서류가 흐리거나 날씨가 없을 수 있다. 재학증명서를 영문으로 다시 발급받아 업로드한다
VS Code에서 Copilot 아이콘이 안 보임	확장 설치 후 GitHub 로그인을 했는지 확인한다
"You don't have access to Copilot"	github.com/settings/copilot 에서 Student 플랜이 활성화되었는지 확인한다

Antigravity

문제	해결
SmartScreen 경고	"추가 정보" → "실행"을 클릭한다. Google 공식 앱이므로 안전하다
로그인 안 됨	브라우저에서 Google 계정에 먼저 로그인한 뒤 다시 시도한다

TRAE

문제	해결
프로젝트 폴더를 못 열거나 파일 수정이 안 됨	앱에서 작업 폴더를 다시 열고(또는 권한 허용), 동일 폴더를 열었는지 확인한다
에이전트가 파일을 못 찾는다고 함	먼저 "현재 폴더 구조를 요약해줘"라고 시켜 컨텍스트를 잡고, 필요한 파일을 명시적으로 지정한다

OpenCode

문제	해결
<code>opencode</code> 명령을 찾을 수 없음	새 터미널을 열어본다. 안 되면 <code>npm install -g opencode-ai@latest</code> 를 다시 실행한다
<code>/models</code> 에 모델이 안 보임	<code>/connect</code> 로 제공자 연결이 되었는지 먼저 확인한다
무료 모델이 없다고 나옴	무료 모델 목록은 수시로 바뀐다. 다른 시간에 다시 확인하거나 Ollama 로컬 모델을 쓴다
VS Code 확장이 자동 설치 안 됨	<code>Ctrl+Shift+P</code> → "Shell Command: Install 'code' command in PATH" 실행 후 재시도

프로젝트 규칙 파일

문제	해결
"이 프로젝트 규칙을 요약해줘"에 규칙 내용이 안 나옴	파일이 작업 폴더 루트 에 있는지 확인한다. 하위 폴더에 둔 <code>AGENTS.md</code> 는 기본 설정으로 읽지 않는다 (VS Code는 <code>chat.useNestedAgentsMdFiles</code> 가 꺼져 있다)
VS Code Copilot이 <code>AGENTS.md</code> 를 무시함	<code>Ctrl + ,</code> 로 설정을 열고 <code>chat.useAgentsMdFile</code> 을 검색해 켜져 있는지 확인한다
Claude Code가 <code>AGENTS.md</code> 를 못 읽음	Claude Code는 <code>AGENTS.md</code> 를 읽지 않는다. <code>CLAUDE.md</code> 에 <code>@AGENTS.md</code> 를 넣었는지 확인한다 (6-3)
규칙을 써 뒀는데 잘 안 지킴	규칙이 추상적이면 지켜지지 않는다. "코드를 깔끔하게"가 아니라 "한 함수는 40줄을 넘기지 않는다"처럼 확인할 수 있는 문장으로 바꾼다
<code>todo.md</code> 를 갱신하지 않음	<code>AGENTS.md</code> 에 "작업을 끝내면 <code>todo.md</code> 에 기록한다"가 들어 있는지 확인한다. 파일만 만들어 두면 읽지 않는다